Sparc4D: A Compact Explicit 4D Representation for Dynamic Scenes

Di Yang[1] Zhihao Li[2] Yanhai Xiong[1,*] Yufei Wang[2,*,†]
[1]College of William & Mary [2]SparcAI Inc.
dyang06@wm.edu zhihao.li@sparclab.ai yxiong05@wm.edu yufei.wang@sparclab.ai

Abstract

We present Sparc4D, a feed-forward autoencoder that encodes a monocular video with known cameras into a compact explicit 4D scene state. Static latent and appearance features are stored once per clip, and spatially anchored temporal slots compress the time-varying features. The slots use a median $5.0\times$ fewer values than per-frame features, with no loss in novel-view quality. A sparse decoder reconstructs 2D Gaussian surfels whose geometry supports both rendering and reprojection of stored source texture. The stored texture keeps the first source frame and compact learned codes for dynamic regions. On random MultiCamVideo windows, the complete state averages 0.40M values per 32-frame clip. At this size, Sparc4D outperforms C4G fine-tuned on the same data while using a $2.3\times$ smaller state. It also reaches higher PSNR than the released MoVieS model, whose output is about 570 times larger, with comparable LPIPS. Training on synthetic scenes also supports zero-shot reconstruction on DyCheck and Neu3D. After domain adaptation, the frozen encoder supports a ManiSkill3 StackCube policy with 81.3% success from 1,000 demonstrations.

1. Introduction

Feed-forward dynamic-scene reconstruction turns monocular video into a representation that can be rendered from new viewpoints. Storing and reusing that representation requires a balance between state size and reconstruction quality. Pixel-aligned Gaussian methods such as MoVieS [27] store one primitive per input pixel, so their state grows with the number of input pixels. Compact Gaussian predictors such as C4G [15] reduce the number of primitives and trade reconstruction fidelity for size. We study how to encode a posed monocular video into a compact explicit 4D scene state while retaining its novel-view appearance.

Video tokenizers [33, 59] compress image sequences, but their pixel-space outputs do not by themselves provide an explicit 3D rendering interface. Sparse 3D autoencoders [52, 53] provide such structure for individual assets. Dynamic scenes require both temporal variation and spatial detail. Per-frame storage repeats static content and grows with the number of observations, while a low-resolution latent limits the appearance detail that each cell can retain. These information types motivate complementary storage paths: shared static features, compressed time-varying features and fine source appearance.

Sparc4D assigns static and dynamic observations different temporal representations. Estimated depth lifts a posed video into sparse 3D observations, which a shared encoder maps to spatial features. Static and far-field features are stored once per clip. Dynamic features are encoded per frame, then pooled into a small number of temporal slots at each occupied spatial cell. Reading these slots at a cell and timestamp recovers the dynamic features needed by the sparse decoder. Sparse occupancy is stored alongside the features to define the decoded surfaces.

Fine appearance is retained through learned appearance codes and stored source texture. A fine-grid appearance code supplements the 32^3 latent. The stored texture keeps the first source frame. Every fourth frame, a learned texture code stores what the state's own rendering of the source view misses in dynamic regions. The decoder reconstructs 2D Gaussian surfels [12], whose geometry supports both explicit rendering and reprojection of the decoded texture. This reprojection follows image-based rendering and depth-image-based video coding [4, 7, 9, 45]. The residual texture code follows conditional video coding [20]. The autoencoder is trained with stored texture applied, and the texture codec is trained afterwards on the frozen autoencoder.

We evaluate reconstruction quality and state size on MultiCamVideo [2], including a C4G model fine-tuned on the same training split. We also test zero-shot transfer to DyCheck [10] and Neu3D [21]. Component ablations show how each part of the state contributes to size and quality. Finally, a manipulation policy trained on frozen encoder features tests their utility for control.

*Corresponding author. †Project lead.
Preprint. Work in progress. 2026 Sep 26th.

The contributions are:

- We introduce Sparc4D, a feed-forward autoencoder that encodes a posed monocular video into a compact explicit 4D scene state rendered through 2D Gaussian surfels.
- Spatially anchored temporal slots are proposed to store the time-varying features of each occupied cell, while static features are shared across the clip.
- On MultiCamVideo, Sparc4D matches or exceeds pixel-aligned and compact Gaussian predictors with 0.40M values per 32-frame clip. It also reconstructs real videos zero-shot and supports manipulation from frozen encoder features.

2. Related Work

Dynamic scene reconstruction. Dynamic radiance fields model time-varying scenes through deformation [35, 36, 38], scene flow [22], or image-based rendering [23]. Gaussian splatting [14] supports deformable and time-varying primitives [32, 51, 57]. Shape of Motion and MoSca regularize motion using low-dimensional bases or scaffolds [19, 49]. These methods optimize a scene-specific representation. We instead learn an encoder and decoder shared across scenes.

Feed-forward reconstruction. Image-based reconstruction networks predict 3D representations in one pass [5, 6, 11, 54, 61]. VGGT and MonST3R estimate geometry and cameras from static or dynamic inputs [47, 60]. For dynamic reconstruction, L4GM and STORM predict time-dependent Gaussians [40, 56]. MoVieS attaches motion to pixel-aligned Gaussians [27], 4DGT predicts space-time Gaussians from monocular video [55], and C4G uses timestamp-conditioned queries to decode a compact Gaussian set [15]. We compare with the latter three using their public weights and additionally fine-tune C4G on our training split at its original output budget. We compare methods by the size of their encoded scene state and its reconstruction quality.

Scene and video compression. Video tokenizers such as MAGVIT-v2 and Cosmos encode image sequences into spatiotemporal latents [33, 59]. TRELLIS, TRELLIS.2, Sparc3D and XCube use structured or hierarchical sparse representations for 3D assets [25, 41, 52, 53]. SceneTok encodes static scenes into tokens decoded by a diffusion model [1]. We adapt the TRELLIS.2 shape autoencoder to dynamic scenes and add a temporal bottleneck based on learned queries [13, 30]. Structured 4D latents have also been used for robot planning [26]; Section 4.5 relates a policy built on Sparc4D features to their StackCube result.

Image-based rendering. View-dependent texture mapping, unstructured lumigraphs and learned image-based rendering use reference images to texture scene geometry [4, 7, 42, 48]. Depth-image-based video coding similarly combines stored images and geometry to synthesize new views [9, 45]. Our stored texture follows this principle, but uses geometry reconstructed from the stored 4D state rather than transmitted depth maps. For evaluation, we also adopt the distinction between observed and unobserved regions motivated by DyCheck [10].

3. Method

Sparc4D stores a clip as shared static features, spatially anchored dynamic slots and stored texture (Fig. 1). The decoder reads this state at an observed time and renders it through explicit surfel geometry. Appendix A.1 summarizes the state and the encoding/decoding procedure.

3.1. Scene observations

The input is a monocular clip of $T = 32$ RGB frames $\{I_t\}$ at 256×256, recorded by one moving *source camera* with known intrinsics and world-to-camera poses. We estimate depth using Depth Anything 3 [28], conditioned on these cameras. A pixel $\mathbf{x}$ is lifted into world coordinates as

$$\mathbf{p}_t(\mathbf{x}) = \mathbf{R}_t^\top \left(D_t(\mathbf{x}) \mathbf{K}^{-1} \tilde{\mathbf{x}} - \mathbf{t}_t \right), \tag{1}$$

where D_t is estimated depth, $(\mathbf{R}_t, \mathbf{t}_t)$ is the camera pose, and $\tilde{\mathbf{x}}$ is the homogeneous pixel coordinate. Each point carries RGB, a depth-derived normal and its position.

Static observations accumulate spatial evidence across the clip, while dynamic observations retain their timestamps. We therefore pool static points and retain dynamic points separately for each frame. To separate them, we mark a point as dynamic when another frame sees through its expected position. Foreground object masks [63] extend these dynamic pixels to whole objects, including intervals when an object briefly stops. Appendix A gives the test and mask parameters.

We voxelize the observations inside a scene-adaptive cube at 512^3 resolution. We contract static points outside the cube into a bounded shell, using the contraction of Mip-NeRF 360 [3] with the L_∞ norm as in Nerfstudio [43]. With cube centre $\mathbf{c}$ and half-size r, a point $\mathbf{x}$ maps to

$$\mathbf{y} = \begin{cases} \tilde{\mathbf{x}}, & \|\tilde{\mathbf{x}}\|_\infty \le 1, \\ \left(2 - \frac{1}{\|\tilde{\mathbf{x}}\|_\infty}\right) \frac{\tilde{\mathbf{x}}}{\|\tilde{\mathbf{x}}\|_\infty}, & \text{otherwise}, \end{cases} \qquad \tilde{\mathbf{x}} = \frac{\mathbf{x} - \mathbf{c}}{r}. \tag{2}$$

The L_∞ norm keeps the shell boundary aligned with the cubic grid. The contracted points are voxelized on a second 512^3 grid over $[-2, 2]^3$. We call the three voxel sets the static, far-field and dynamic *layers* of the scene. Pixels without surfels receive the source video's mean colour.

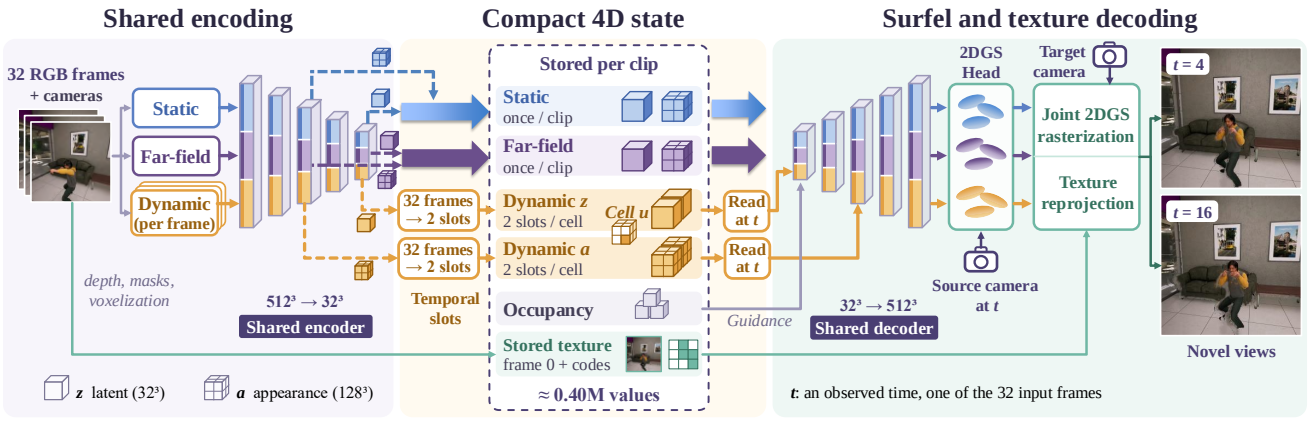


Figure 1. **Overview of Sparc4D.** *Shared encoding:* a posed monocular clip is lifted with estimated depth and voxelized into static, far-field and per-frame dynamic layers. A shared sparse encoder maps each layer from 512^3 to a 32^3 latent $\mathbf{z}$ and a 128^3 appearance code $\mathbf{a}$. *Compact 4D state:* static and far-field features are stored once per clip. Temporal slots compress the per-frame dynamic latents and appearance codes of each cell u into two slots. Sparse occupancy and stored texture complete the state, which averages 0.40M values per 32-frame clip. *Decoding:* at an observed time t, the slots are read out and the shared decoder upsamples the features along the stored occupancy. The 2DGS head emits surfels oriented by the source camera at t. All layers are rasterized jointly in the target camera, and stored texture is reprojected onto the surfels.

Each occupied voxel stores mean RGB, mean normal and a within-voxel point offset. Octant colour residuals retain sub-voxel variation before encoding.

3.2. Shared static features and temporal slots

We initialize the sparse encoder–decoder from the TRELLIS.2 shape autoencoder [53]. It maps a 512^3 sparse voxel grid to a 32-channel latent on a 32^3 grid. The static, far-field and per-frame dynamic layers are encoded as separate batches with the same network weights. The static and far-field latents are stored once per clip. The decoder uses the input occupancy hierarchy for sparse upsampling; this hierarchy is part of the stored state.

Temporal slots compress the dynamic features at each fixed spatial location. A *cell* is one position of the 32^3 latent grid, as illustrated by cell u in Fig. 1. Let $\mathcal{U}$ be the union of dynamic cells over the clip, and $\mathbf{z}_{u,t}$ the latent of cell u at time t. We compress the available per-frame features into two slots per cell. We obtain them by multihead attention pooling with two learnable seed vectors [17], followed by cross-cell mixing and projection:

$$\begin{aligned} \mathbf{k}_{u,t} &= \mathbf{W}\mathbf{z}_{u,t} + \phi_t(t) + \phi_p(u), \\ \mathbf{A}_u &= \mathrm{Attn}(\mathbf{Q} + \phi_p(u), \{\mathbf{k}_{u,t}\}_{t\in\mathcal{T}_u}), \\ \{\mathbf{S}_u\}_{u\in\mathcal{U}} &= \mathrm{Proj}(\mathrm{Mix}(\{\mathbf{A}_u\}_{u\in\mathcal{U}})), \\ \hat{\mathbf{z}}_{u,t} &= \mathrm{Dec}(\phi_t(t) + \phi_p(u), \mathbf{S}_u). \end{aligned} \tag{3}$$

Here $\mathcal{T}_u$ contains the frames in which cell u is occupied, and ϕ_t, ϕ_p embed time and position. In $\mathrm{Attn}(\cdot, X)$, the first argument provides the queries, and learned projections of X provide the keys and values. Two learned queries $\mathbf{Q}$ first pool features within each cell. Self-attention then mixes the slots across cells, and a projection reduces each slot to 32 channels. To recover a latent, a query at (u, t) attends to the two stored slots of u. The decoded latent $\hat{\mathbf{z}}_{u,t}$ replaces $\mathbf{z}_{u,t}$ in the sparse decoder. The stored dynamic latent contains $2|\mathcal{U}|$ vectors, each with 32 channels.

3.3. Appearance and surfel decoding

A fine-grid appearance code retains spatial detail beyond the 32^3 latent. Each latent cell spans 16^3 input voxels; we supplement it with a 4-channel code on the 128^3 grid. For encoder and decoder features $\mathbf{e}_u$ and $\mathbf{g}_u$ at this resolution,

$$\mathbf{a}_u = \mathbf{W}_e\,\mathrm{LN}(\mathbf{e}_u), \qquad \mathbf{g}_u \leftarrow \mathbf{g}_u + \gamma\mathbf{W}_d\mathbf{a}_u, \tag{4}$$

where LN is layer normalization and γ is a fixed gain. Static and far-field appearance codes are shared across the clip. An independent temporal module applies Eq. 3 to dynamic appearance, storing two 4-channel slots per cell in the 128^3 temporal union. Its readout at an occupied cell and time is $\hat{\mathbf{a}}_{u,t}$.

Each decoded voxel $\mathbf{v}$ emits four 2D Gaussian surfels (2DGS) [12, 37] with degree-1 spherical-harmonics colour. For each surfel, the 2DGS head predicts residuals $(\boldsymbol{\delta}, \boldsymbol{\ell}, \hat{\mathbf{q}}, \omega)$ from a camera-facing base configuration defined by the source camera centre $\mathbf{c}_t^{\mathrm{src}}$ at time t:

$$\begin{aligned} \boldsymbol{\mu} &= \mathbf{o} + h\left(\mathbf{v} + \tfrac{1}{2} + \tanh\boldsymbol{\delta}\right), & \alpha &= \sigma(\omega + 2), \\ \mathbf{s} &= \mathrm{cap}\left(\rho\,\frac{\mathrm{softplus}(\boldsymbol{\ell}+b)}{\mathrm{softplus}(b)}\right), & \rho &= \frac{0.55\,\|\mathbf{c}_t^{\mathrm{src}} - \bar{\mathbf{v}}\|}{f}, \\ \mathbf{q} &= \mathrm{norm}\left(\hat{\mathbf{q}} + \mathbf{1}_w\right) \otimes \mathbf{q}_{\mathrm{face}}\left(\mathbf{c}_t^{\mathrm{src}} - \bar{\mathbf{v}}\right). \end{aligned} \tag{5}$$

Here $\boldsymbol{\mu}$, $\mathbf{s}$, $\mathbf{q}$ and α are the surfel centre, scales, rotation quaternion and opacity. The cube origin is $\mathbf{o}$, the voxel size is h, and the voxel centre is $\bar{\mathbf{v}}$. The footprint ρ is the world-space size of 0.55 pixels at focal length f. We set $b = \log(e-1)$ and $\mathrm{cap}(s) = s\, s_{\max}/(s+s_{\max})$, where $s_{\max}$ is four voxels. The identity quaternion is $\mathbf{1}_w$, and $\mathbf{q}_{\mathrm{face}}(\mathbf{n})$ turns the surfel normal towards $\mathbf{n}$. With a zero-initialized output layer, each voxel starts as a camera-facing disk of about one pixel. Far-field surfels are decoded in contracted coordinates and mapped back to world space by the inverse contraction. The surfels of the three layers are rasterized together with gsplat [58]. Dynamic decoding uses the occupancy of frame t, so t is one of the 32 input timestamps (an *observed time*).

3.4. Stored texture

Stored texture preserves source detail alongside the learned reconstruction of surfaces and appearance. We keep the first source frame in full. Every fourth frame τ, a texture code stores the dynamic region. The dynamic region is where the decoded dynamic layer's source-view opacity exceeds 0.5, dilated by two pixels.

We render the decoded state at the source camera at τ and reproject the first frame onto it, giving R_τ. A convolutional encoder maps I_τ, R_τ and the rendered opacity to a 16-channel code for each 8×8-pixel tile. Only tiles overlapping the dynamic region are stored. The decoder recomputes these tiles from the decoded geometry. The decoder predicts a residual from the code and R_τ, giving $\hat{I}_\tau = R_\tau + D(\mathrm{code}, R_\tau)$. This follows conditional video coding [20]. The codec is trained on the frozen autoencoder with L_1 and LPIPS losses inside the dynamic region.

For a target camera c at time t, rasterization gives a colour image and median depth $\hat{d}_c$. We back-project a target pixel using $\hat{d}_c$ and project the resulting point into a stored frame. If its projected depth agrees with the depth rendered in that stored frame's source view, we use the sampled stored colour. Otherwise, or if the point falls outside the stored region, we retain the surfel colour. Both depth maps are rendered from the decoded state.

Static pixels use the first source frame. Dynamic pixels use the two decoded texture frames $\hat{I}_\tau$ that bracket t. Each frame passes the same projection and depth test, and the accepted colours are blended by temporal distance. Stored texture changes only colour; geometry and coverage come from the surfels. The projection and consistency test are specified in Appendix A.4.

3.5. Training

For each training sample, we select a 32-frame window and one source camera. Its frames form the input. We supervise frame 0, a random time t and the two texture frames that bracket t. At each of these times we render the source view and four of the remaining nine cameras. The image loss combines L_1, SSIM and LPIPS and is computed after stored texture is applied; replaced pixels supervise only opacity. During autoencoder training, the texture frames use source pixels instead of decoded codes. Depth and normal losses supervise geometry, while a temporal loss matches inter-frame image differences. The temporal slots also receive feature-reconstruction losses. Occupancy and KL losses follow TRELLIS.2 [16, 53].

We initialize the encoder and decoder from the pre-trained TRELLIS.2 shape autoencoder. We first train the 4D autoencoder with per-frame dynamic features for 30000 steps. We then add the temporal slots and train them with the decoder for 10000 steps, keeping the encoder frozen. In the last stage, we introduce the appearance codes, stored texture and the far-field shell, and train the full autoencoder for 30000 steps. The texture codec is then trained on the frozen autoencoder for 6000 steps. Appendix B gives the full objective, optimization settings, and ablation protocol.

3.6. State size

We count the stored features, sparse occupancy, background colour and stored texture. The unit is a *32-bit-equivalent value*: feature values count as one each, while occupancy entropy in bits is divided by 32. Each 8-bit RGB channel of the first frame counts as one value, and each texture-code value counts as one. Shared network weights and externally supplied cameras are excluded.

Let $\mathcal{C}$ denote the static and far-field cells and $\mathcal{U}$ the dynamic union cells on the 32^3 grid. Their 128^3 counterparts store appearance. With occupancy cost B_{occ} bits and the set $\mathcal{T}$ of stored code tiles over the refresh frames, the reported size is

$$N = 32|\mathcal{C}| + 64|\mathcal{U}| + 4|\mathcal{C}^{128}| + 8|\mathcal{U}^{128}| + B_{\mathrm{occ}}/32 + 3 + 3 \cdot 256^2 + 16|\mathcal{T}|. \quad (6)$$

Occupancy is estimated from octree symbols. For dynamic occupancy, we use the smaller estimate from independent frames or consecutive-frame XOR masks. Appendix C details this accounting for all compared methods. Table 11 separates the learned state from its stored texture.

4. Experiments

We first evaluate novel-view reconstruction and state size. Ablations then isolate the components of Sparc4D. Finally, real-video transfer and robotic control test the learned representation beyond synthetic reconstruction.

4.1. Data and protocol

Dataset and split. MultiCamVideo (MCV) [2] contains 3,400 Unreal Engine scenes. Each scene is rendered at four focal lengths by ten synchronized cameras for 81 frames

Method	Trained on MCV	#Floats	Full image PSNR↑	Full image SSIM↑	Full image LPIPS↓	AbsRel↓	Co-visible PSNR↑	Co-visible SSIM↑	Co-visible LPIPS↓	Uniform fill PSNR↑	Uniform fill SSIM↑	Uniform fill LPIPS↓
MoVieS	–	228M	*17.26*	*0.512*	**0.366**	0.615 (0.272)	*18.55*	*0.530*	*0.296*	*16.43*	*0.498*	*0.411*
4DGT	–	26.9M	14.35	0.403	0.550	0.640 (0.370)	15.14	0.411	0.538	14.49	0.412	0.546
C4G	–	*0.92M*	13.64	0.369	0.596	0.543 (0.450)	14.76	0.394	0.566	14.21	0.393	0.572
C4G	✓	*0.92M*	16.51	0.466	0.584	*0.356 (0.346)*	17.14	0.470	0.561	15.65	0.444	0.564
Sparc4D	✓	**0.40M**	**17.51**	**0.524**	*0.367*	**0.270 (0.179)**	**19.16**	**0.548**	**0.287**	**16.66**	**0.513**	**0.403**

Table 1. **Novel-view synthesis on MultiCamVideo** (404 random-window test clips, 9 target cameras × 32 frames, 256^2). Best and second-best results are shown in bold and underlined italics, respectively. Sparc4D includes stored texture. #Floats counts the fewest stored 32-bit values that render any input timestamp (Eq. 6 for ours); shared network weights are excluded. C4G is shown with released weights and after fine-tuning for 30,000 steps on the MCV training split. AbsRel uses DA3 pseudo ground truth, also our input depth; parentheses give median-scale-aligned error. Co-visible metrics use the shared visibility mask. Uniform fill assigns the same colour outside that mask for all methods.



Figure 2. **Qualitative comparison on MultiCamVideo.** Top: first-32 protocol. Bottom: random-window protocol. Orange and blue boxes mark dynamic and static 64^2 crops, enlarged below at identical coordinates for all methods. C4G is fine-tuned on the MultiCamVideo training split; the other baselines use released weights. Camera distance is shown in metres.

at 15 fps. A scene–focal-length pair forms one clip, giving 13,600 clips. Frames are resized to 256^2.

The held-out test set contains 101 scenes, selected by clustering DINOv2 scene features [34], with all four focal lengths of each scene, giving 404 clips. All MultiCamVideo results in this paper use this test set. Appendix F.1 gives the construction details.

Windows and cameras. The cameras coincide at the start of each video and then move apart. We therefore evaluate two window placements. The *first-32* protocol uses frames 0–31. The *random-window* protocol fixes one randomly sampled start in $[0, 49]$ for each clip. The source camera is fixed by the clip's test-list index modulo ten; the other nine cameras are targets. All methods render every target at all 32 timestamps, giving 116,352 target images per protocol.

Metrics. We report image-averaged PSNR, SSIM [50] and AlexNet-LPIPS [62]. Alongside full-image scores, we evaluate pixels classified as observed by a method-

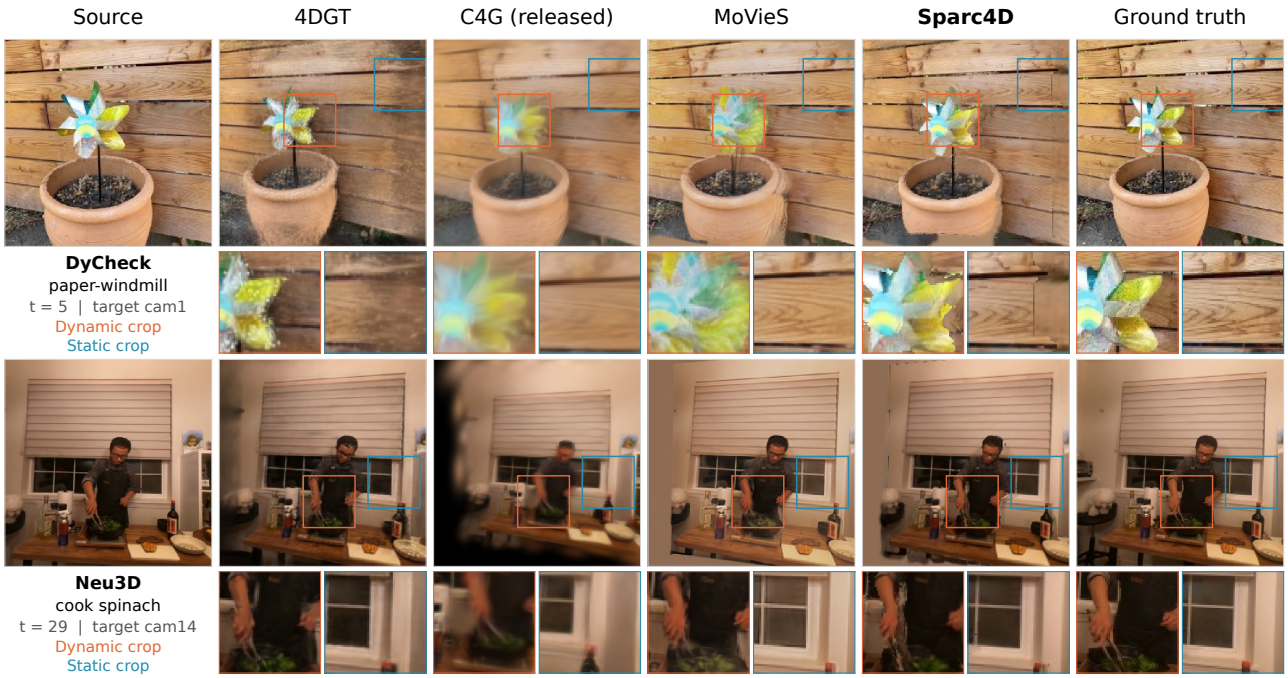


Figure 3. **Zero-shot comparison on real videos.** Top: DyCheck. Bottom: Neu3D. All baselines use released weights, and no method is fine-tuned on these datasets. Boxes and crops follow Fig. 2.

Table 2. **Component ablation** (404 random-window clips). Each row removes or replaces one component of Sparc4D; all rows use the same trained weights. Per-frame features store dynamic latent and appearance features at every timestamp instead of temporal slots. Without the far-field shell, static points outside the scene cube are discarded. Without reuse across time, each decoded texture frame colours only its own timestamp. Without the depth test, a reprojected colour is accepted whenever the point projects into the stored frame. #Floats is the mean state size per 32-frame clip (Eq. 6). Dyn. LPIPS is computed in the dynamic region (Appendix F.2).

Configuration	#Floats↓	PSNR↑	SSIM↑	LPIPS↓	Dyn. LPIPS↓
Sparc4D	0.40M	17.51	0.524	0.367	0.264
Per-frame features	0.67M	17.50	0.524	0.367	0.265
w/o far-field shell	0.39M	16.46	0.500	0.416	0.305
w/o stored texture	0.16M	16.99	0.496	0.466	0.346
w/o texture codes	0.35M	17.42	0.523	0.381	0.294
w/o reuse across time	0.40M	17.44	0.523	0.377	0.287
w/o depth test	0.40M	17.24	0.517	0.370	0.278

Table 3. **Temporal feature storage** over the 404 random-window test clips. Sizes are mean 32-bit-equivalent values per clip. Compression factors are computed per clip and reported as their median and mean. Total state includes stored texture.

Component	Per-frame	Slots	Median	Mean
Dynamic latent	182.1K	22.7K	8.8×	8.7×
Dynamic appearance	172.5K	56.7K	3.5×	3.8×
Time-varying features	354.5K	79.4K	5.0×	5.2×
Total state	0.67M	0.40M	1.6×	1.6×

Table 4. **Number of temporal slots** K on MultiCamVideo (404 test clips, full image). All variants start from the same checkpoint and are trained for 12k steps without appearance codes; only K differs. Dynamic-slot size is the mean number of 32-bit values per clip. ΔPSNR is the per-clip paired difference to K=2 (mean ± 95% CI).

K	Dyn. slot #Floats	PSNR↑	SSIM↑	LPIPS↓	ΔPSNR
2	**21.1K**	16.76	*0.492*	*0.475*	–
3	*31.7K*	*16.78*	**0.493**	*0.475*	+0.02 ± 0.01
10	105.6K	**16.80**	**0.493**	**0.474**	+0.04 ± 0.01

independent visibility mask. The mask uses estimated depth and foreground segmentation; dynamic foreground requires same-time source visibility. A third setting, *uniform fill*, assigns the same source-mean colour to all methods outside the mask. These settings separate reconstruction on observed regions from differences in unobserved-region rendering. Appendix F.2 specifies the masks and masked met-

Table 5. **Static and dynamic regions**, first-32 protocol. Regions are defined in Appendix F.2.

	Dynamic		Static
Method	PSNR↑	LPIPS↓	PSNR↑
4DGT	16.19	0.345	18.56
C4G	14.32	0.434	15.97
MoVieS	*19.02*	**0.150**	*21.79*
w/o stored texture	17.41	0.275	21.49
w/o texture codes	18.18	0.202	**25.28**
Sparc4D	**19.21**	*0.153*	**25.28**

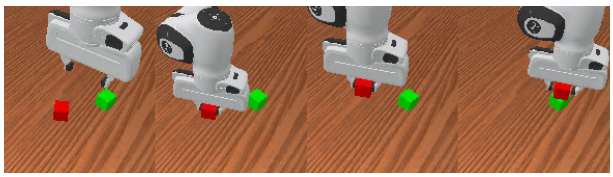


Figure 4. **StackCube manipulation.** A successful rollout of the policy trained on 1,000 demonstrations, shown from one fixed camera. The robot grasps the red cube and places it on the green cube.

rics.

Baselines and comparison scope. We evaluate MoVieS [27], 4DGT [55] and C4G [15] with released weights. Their training data differ from ours, so these models serve as zero-shot references. For a same-data comparison at the closest representation size, we fine-tune C4G on the MultiCamVideo training split for 30,000 steps, keeping its 2,048-Gaussian output budget. Sparc4D is trained on the same split. All methods render at 256^2; input settings and coordinate alignment are given in Appendix F.3. #Floats counts the fewest 32-bit values from which a method renders any of the 32 input timestamps; shared network weights are excluded (Appendix C).

4.2. Novel-view reconstruction

On random windows, Sparc4D reaches 17.51 dB with a 0.40M-value state (Table 1). MoVieS obtains 17.26 dB; the paired per-clip difference is $+0.24 \pm 0.18$ dB (95% CI). On first-32, Sparc4D reaches 22.17 dB versus 20.40 dB for MoVieS; the paired difference is $+1.77 \pm 0.28$ dB.

On co-visible pixels, Sparc4D obtains 19.16 dB and 0.287 LPIPS; MoVieS obtains 18.55 dB and 0.296. The paired LPIPS difference is -0.009 ± 0.008. Uniform-fill evaluation preserves the PSNR ordering. With a 2.3× smaller state than MCV-fine-tuned C4G (0.40M versus 0.92M), Sparc4D has higher full-image PSNR and lower LPIPS. Section 4.3 ablates the components of Sparc4D.

Figure 2 compares Sparc4D with the baselines under both protocols. All methods share the target view and crop locations.

4.3. Ablation study

Components. Table 2 removes or replaces one component of Sparc4D at a time, keeping the trained weights fixed. Per-frame dynamic features enlarge the state from 0.40M to 0.67M values without improving quality. Removing the far-field shell saves only 10K values but lowers PSNR by 1.04 dB. Without stored texture, PSNR drops from 17.51 to 16.99 dB and LPIPS rises from 0.367 to 0.466. The first frame alone recovers most of this loss. The texture codes add 43K values and lower dynamic-region LPIPS from 0.294 to 0.264. Using each decoded texture frame only at its own timestamp raises dynamic-region LPIPS to 0.287. Accepting reprojected colours without the depth test lowers PSNR by 0.27 dB.

Geometry versus stored texture. Stored texture changes only colour, so we keep the stored pixels fixed and replace the decoded geometry (Appendix D.2). With the same stored pixels, rendering the input voxels directly requires 3.6× more values than Sparc4D and lowers PSNR by 0.38 dB. Reprojecting the stored frames with their own DA3 depth instead lowers PSNR by 0.51 dB.

Temporal compression. Temporal slots reduce the mean time-varying feature count from 354.5K to 79.4K values per clip (Table 3). The per-clip compression factor has a median of 5.0×: 8.8× for the dynamic latent and 3.5× for the dynamic appearance. Appendix C gives the storage formulas. With the same or more storage, fixed temporal models reconstruct the dynamic latents with 1.8–4.6× higher error than the slots (Appendix D.1). Without stored texture, they also render dynamic regions 0.04–0.11 dB worse. With stored texture, per-cell means and DCT or linear models render within 0.02 dB of the slots. This indicates that per-frame occupancy and stored texture carry most of the temporal change.

Design parameters. Table 4 varies the number of temporal slots. Ten slots raise PSNR by only 0.04 dB over two slots and store five times as many dynamic-slot values. Appendix B.1 varies the refresh interval of the texture codes. Refreshing the codes at every frame instead of every fourth frame adds 0.14M values and 0.03 dB.

Static and dynamic reconstruction. On first-32, stored texture matters most in static regions: it raises PSNR from 21.49 to 25.28 dB, above the 21.79 dB of MoVieS (Table 5). In dynamic regions, the texture codes raise PSNR from

Method	Training data	#Floats	DyCheck mPSNR↑	DyCheck mSSIM↑	DyCheck mLPIPS↓	Neu3D (9 targets) PSNR↑	Neu3D (9 targets) SSIM↑	Neu3D (9 targets) LPIPS↓	Neu3D cam00 PSNR↑
MoVieS	8 public datasets (∼174K scenes)	228M	**13.95**	*0.345*	**0.488**	15.61	0.445	0.304	17.01
4DGT	∼300 h real monocular video	26.9M	11.84	0.277	0.599	**17.33**	**0.530**	**0.281**	**19.51**
C4G	Spring, Kubric, RE10K	*0.92M*	13.52	**0.351**	0.634	11.78	0.285	0.588	13.41
C4G (MCV fine-tuned)	+ MultiCamVideo	*0.92M*	13.34	0.340	0.747	15.75	0.443	0.579	16.21
Sparc4D	MultiCamVideo (synthetic)	**0.40M**	*13.88*	0.319	*0.547*	*16.58*	*0.465*	*0.293*	*18.58*

Table 6. **Zero-shot generalization to real videos.** Best and second-best results are shown in bold and underlined italics. None of the methods is fine-tuned on DyCheck or Neu3D. DyCheck uses official co-visibility evaluation; Neu3D reports nine target cameras and cam00 separately. #Floats repeats the MCV reference size for context rather than a measured real-data average.

Table 7. **Stored texture on real videos.** All rows use the same model weights and inputs. DyCheck uses official masked metrics; Neu3D uses full-image metrics over nine target cameras.

Configuration	DyCheck mPSNR↑	DyCheck mLPIPS↓	Neu3D PSNR↑	Neu3D LPIPS↓
w/o stored texture	14.23	0.610	16.62	0.389
w/o texture codes	14.08	0.563	16.54	0.321
Sparc4D	13.88	0.547	16.58	0.293

Table 8. **StackCube success and scene size.** Tokens are active cells per timestamp; d is their feature dimension. Protocols and counting details are in Appendix I.

Method	Demos	Tokens	d	Success
Li et al. [26]	1,000	∼5,700	8	84.0%
Sparc4D features	1,000	661	32	81.3%
+ additional demos	3,000	661	32	84.0%

18.18 to 19.21 dB, on par with MoVieS (19.02 dB; LPIPS 0.153 versus 0.150).

Appendix H shows that Sparc4D preserves the DA3 depth structure more closely than the evaluated baselines.

4.4. Transfer to real videos

We evaluate zero-shot transfer to DyCheck iPhone [10] and Neu3D [21] (Table 6) without real-data fine-tuning. DyCheck uses official co-visibility masks, whereas Neu3D uses full-image metrics; Appendix F.4 and Appendix F.5 give the full protocols.

On DyCheck, Sparc4D reaches 13.88 dB mPSNR, close to MoVieS at 13.95 dB. On Neu3D, it reaches 16.58 dB across nine target cameras and 18.58 dB on cam00.

Table 7 isolates the effect of stored texture using identical model weights and inputs. Stored texture lowers LPIPS from 0.610 to 0.547 on DyCheck and from 0.389 to 0.293 on Neu3D. Figure 3 shows novel views from both datasets, and Appendix G gives more examples.

4.5. Downstream manipulation

We evaluate the learned scene features on StackCube in ManiSkill3 [44], where a robot must grasp the red cube and stack it on the green cube (Fig. 4). After adapting the autoencoder to ManiSkill scenes, we freeze its encoder and train a flow-matching action policy on static features and the dynamic latents of the three most recent frames. Four calibrated RGB-D views provide the observations; robot masks and causal SAM 2 tracking [39] separate the static and dynamic layers. This control interface uses per-frame latents without temporal-slot compression or stored texture.

With 1,000 demonstrations, the policy reaches 81.3% success, close to the 84.0% reported by Li et al. [26]; adding 2,000 demonstrations raises it to 84.0% (Table 8). Our scene representation averages 426 static and 236 dynamic tokens, with the static features shared across the observation history. The rollout in Fig. 4 illustrates control from these features.

5. Conclusion

Sparc4D represents a monocular video through shared static features, spatially anchored temporal slots and learned texture codes. This split yields an explicit 4D state averaging 0.40M values per 32-frame clip. Temporal slots reduce time-varying feature storage by a median 5.0× with no loss in novel-view quality. Texture codes keep source detail at 14% of the values of raw dynamic pixels. Synthetic training transfers to DyCheck and Neu3D, and adapted, frozen encoder features support StackCube manipulation. Extending the decoder to continuous time, including dynamic occupancy between observed times, is future work.

References

[1] Mohammad Asim, Christopher Wewer, and Jan Eric Lenssen. SceneTok: A compressed, diffusable token space for 3D scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 2

[2] Jianhong Bai, Menghan Xia, Xiao Fu, Xintao Wang, Lianrui Mu, Jinwen Cao, Zuozhu Liu, Haoji Hu, Xiang Bai, Pengfei

Wan, and Di Zhang. ReCamMaster: Camera-controlled generative rendering from a single video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. 1, 4

[3] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5470–5479, 2022. 2

[4] Chris Buehler, Michael Bosse, Leonard McMillan, Steven Gortler, and Michael Cohen. Unstructured lumigraph rendering. In *Proceedings of SIGGRAPH*, pages 425–432, 2001. 1, 2

[5] David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelSplat: 3D Gaussian splats from image pairs for scalable generalizable 3D reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[6] Yuedong Chen, Haofei Xu, Chuanxia Zheng, Bohan Zhuang, Marc Pollefeys, Andreas Geiger, Tat-Jen Cham, and Jianfei Cai. MVSplat: Efficient 3D Gaussian splatting from sparse multi-view images. In *European Conference on Computer Vision (ECCV)*, 2024. 2

[7] Paul E. Debevec, Camillo J. Taylor, and Jitendra Malik. Modeling and rendering architecture from photographs: A hybrid geometry- and image-based approach. In *Proceedings of SIGGRAPH*, pages 11–20, 1996. 1, 2

[8] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. 16

[9] Christoph Fehn. Depth-image-based rendering (DIBR), compression, and transmission for a new approach on 3D-TV. In *Stereoscopic Displays and Virtual Reality Systems XI*, pages 93–104, 2004. 1, 2

[10] Hang Gao, Ruilong Li, Shubham Tulsiani, Bryan Russell, and Angjoo Kanazawa. Monocular dynamic view synthesis: A reality check. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1, 2, 8

[11] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. LRM: Large reconstruction model for single image to 3D. In *International Conference on Learning Representations (ICLR)*, 2024. 2

[12] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian splatting for geometrically accurate radiance fields. In *SIGGRAPH 2024 Conference Papers*, 2024. 1, 3, 13

[13] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, 2021. 2

[14] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42 (4), 2023. 2

[15] Mungyeom Kim, Minkyeong Jeon, Honggyu An, Jaewoo Jung, Hyuna Ko, Jisang Han, Hyeonseo Yu, Donghwan Shin, Sunghwan Hong, Takuya Narihira, et al. Learning global motion with compact Gaussians for feed-forward 4D reconstruction. *arXiv preprint arXiv:2605.31595*, 2026. 1, 2, 7

[16] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014. 4, 13

[17] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International Conference on Machine Learning (ICML)*, 2019. 3

[18] Junoh Lee, Chang-Yeon Won, Hyunjun Jung, Inhwan Bae, and Hae-Gon Jeon. Fully explicit dynamic Gaussian splatting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 14

[19] Jiahui Lei, Yijia Weng, Adam W. Harley, Leonidas Guibas, and Kostas Daniilidis. MoSca: Dynamic Gaussian fusion from casual videos via 4D motion scaffolds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 2

[20] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 1, 4

[21] Tianye Li, Mira Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, Steven Lovegrove, Michael Goesele, Richard Newcombe, and Zhaoyang Lv. Neural 3D video synthesis from multi-view video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 1, 8, 16

[22] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2

[23] Zhengqi Li, Qianqian Wang, Forrester Cole, Richard Tucker, and Noah Snavely. DynIBaR: Neural dynamic image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2

[24] Zhan Li, Zhang Chen, Zhong Li, and Yi Xu. Spacetime Gaussian feature splatting for real-time dynamic view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 14

[25] Zhihao Li, Yufei Wang, Heliang Zheng, Yihao Luo, and Bihan Wen. Sparc3d: Sparse representation and construction for high-resolution 3d shapes modeling. *Advances in Neural Information Processing Systems*, 38:118582–118600, 2026. 2

[26] Zhiyi Li, Peilin Wu, Xiaoshen Han, Ruojin Cai, and Yilun Du. Structured 4d latent predictive model for robot planning. *arXiv preprint arXiv:2607.01166*, 2026. 2, 8, 16

[27] Chenguo Lin, Yuchen Lin, Panwang Pan, Yifan Yu, Tao Hu, Honglei Yan, Katerina Fragkiadaki, and Yadong Mu. MoVieS: Motion-aware 4D dynamic view synthesis in one second. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 1, 2, 7

[28] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025. 2

[29] Youtian Lin, Zuozhuo Dai, Siyu Zhu, and Yao Yao. Gaussian-Flow: 4D reconstruction with dynamic 3D Gaussian particle. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 14

[30] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2

[31] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. 13

[32] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3D Gaussians: Tracking by persistent dynamic view synthesis. In *International Conference on 3D Vision (3DV)*, 2024. 2

[33] NVIDIA. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025. 1, 2

[34] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 5

[35] Keunhong Park, Utkarsh Sinha, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Steven M. Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. 2

[36] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. HyperNeRF: A higher-dimensional representation for topologically varying neural radiance fields. *ACM Transactions on Graphics*, 40(6), 2021. 2

[37] Hanspeter Pfister, Matthias Zwicker, Jeroen van Baar, and Markus Gross. Surfels: Surface elements as rendering primitives. In *Proceedings of SIGGRAPH*, pages 335–342, 2000. 3

[38] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2

[39] Nikhila Ravi et al. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 8

[40] Jiawei Ren, Kevin Xie, Ashkan Mirzaei, Hanxue Liang, Xiaohui Zeng, Karsten Kreis, Ziwei Liu, Antonio Torralba, Sanja Fidler, Seung Wook Kim, and Huan Ling. L4GM: Large 4D Gaussian reconstruction model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 2

[41] Xuanchi Ren, Jiahui Huang, Xiaohui Zeng, Ken Museth, Sanja Fidler, and Francis Williams. XCube: Large-scale 3D generative modeling using sparse voxel hierarchies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[42] Gernot Riegler and Vladlen Koltun. Free view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020. 2

[43] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Justin Kerr, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, David McAllister, and Angjoo Kanazawa. Nerfstudio: A modular framework for neural radiance field development. In *SIGGRAPH 2023 Conference Papers*, 2023. 2

[44] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. ManiSkill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2025. 8

[45] Gerhard Tech, Ying Chen, Karsten Müller, Jens-Rainer Ohm, Anthony Vetro, and Ye-Kui Wang. Overview of the multiview and 3D extensions of high efficiency video coding. *IEEE Transactions on Circuits and Systems for Video Technology*, 26(1):35–49, 2016. 1, 2

[46] Chaoyang Wang, Ben Eckart, Simon Lucey, and Orazio Gallo. Neural trajectory fields for dynamic novel view synthesis. *arXiv preprint arXiv:2105.05994*, 2021. 14

[47] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 2

[48] Qianqian Wang, Zhicheng Wang, Kyle Genova, Pratul P. Srinivasan, Howard Zhou, Jonathan T. Barron, Ricardo Martin-Brualla, Noah Snavely, and Thomas Funkhouser. IBRNet: Learning multi-view image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2

[49] Qianqian Wang, Vickie Ye, Hang Gao, Jake Austin, Zhengqi Li, and Angjoo Kanazawa. Shape of motion: 4D reconstruction from a single video. *arXiv preprint arXiv:2407.13764*, 2024. 2, 14

[50] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. 5

[51] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[52] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3D latents for scalable and versatile 3D generation. *arXiv preprint arXiv:2412.01506*, 2024. 1, 2

[53] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, Ruicheng Wang, Zelong Lv, Yu Deng, Hongyuan Zhu, Yue Dong, Hao

Zhao, Nicholas Jing Yuan, and Jiaolong Yang. Native and compact structured latents for 3D generation. *Tech report*, 2025. 1, 2, 3, 4, 13

[54] Haofei Xu, Songyou Peng, Fangjinhua Wang, Hermann Blum, Daniel Barath, Andreas Geiger, and Marc Pollefeys. DepthSplat: Connecting Gaussian splatting and depth. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 2

[55] Zhen Xu, Zhengqin Li, Zhao Dong, Xiaowei Zhou, Richard Newcombe, and Zhaoyang Lv. 4DGT: Learning a 4D Gaussian transformer using real-world monocular videos. *arXiv preprint arXiv:2506.08015*, 2025. 2, 7

[56] Jiawei Yang, Jiahui Huang, Yuxiao Chen, Yan Wang, Boyi Li, Yurong You, Apoorva Sharma, Maximilian Igl, Peter Karkus, Danfei Xu, Boris Ivanovic, Yue Wang, and Marco Pavone. STORM: Spatio-temporal reconstruction model for large-scale outdoor scenes. In *International Conference on Learning Representations (ICLR)*, 2025. 2

[57] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3D Gaussians for high-fidelity monocular dynamic scene reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2

[58] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for Gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025. 4

[59] Lijun Yu, José Lezama, Nitesh B. Gundavarapu, Luca Versari, Kihyuk Sohn, David Minnen, Yong Cheng, Agrim Gupta, Xiuye Gu, Alexander G. Hauptmann, Boqing Gong, Ming-Hsuan Yang, Irfan Essa, David A. Ross, and Lu Jiang. Language model beats diffusion – tokenizer is key to visual generation. In *International Conference on Learning Representations (ICLR)*, 2024. 1, 2

[60] Junyi Zhang, Charles Herrmann, Junhwa Hur, Varun Jampani, Trevor Darrell, Forrester Cole, Deqing Sun, and Ming-Hsuan Yang. MonST3R: A simple approach for estimating geometry in the presence of motion. In *International Conference on Learning Representations (ICLR)*, 2025. 2

[61] Kai Zhang, Sai Bi, Hao Tan, Yuanbo Xiangli, Nanxuan Zhao, Kalyan Sunkavalli, and Zexiang Xu. GS-LRM: Large reconstruction model for 3D Gaussian splatting. In *European Conference on Computer Vision (ECCV)*, 2024. 2

[62] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 5

[63] Peng Zheng, Dehong Gao, Deng-Ping Fan, Li Liu, Jorma Laaksonen, Wanli Ouyang, and Nicu Sebe. Bilateral reference for high-resolution dichotomous image segmentation. *CAAI Artificial Intelligence Research*, 3, 2024. 2, 12

A. Observation preprocessing and decoding

A.1. Stored state and encoding/decoding procedure

Table 9 summarizes the information retained after encoding. The latent and appearance grids use independent temporal modules; a cell indexes a fixed spatial position on its respective grid. Their attention modules operate at width 256, then project to the stored channel widths of 32 and 4. Source and target cameras are supplied externally. Cube origins and scales define the grid-to-world transforms; their metadata cost is omitted from the size ledger in Appendix C.

Encode. Given the RGB clip and its source cameras:

1. Estimate depth, lift the observations, separate static and dynamic points, and voxelize the three scene layers.
2. Apply the shared encoder to the static, far-field and per-frame dynamic layers. Collect latent features, appearance codes and the sparse occupancy hierarchies.
3. Retain static and far-field codes once. Independently pool dynamic latent and appearance features into two slots per cell on their temporal unions.
4. Decode the source views at every fourth timestamp to obtain the dynamic region and the reference render R_τ. Retain the full first RGB frame and encode the dynamic region of each of these frames into a texture code (Sec. 3.4).
5. Retain the background colour alongside the feature state. The cube layout defines its world-coordinate interpretation.

Decode. Given the stored state, cameras and an observed timestamp t:

1. Read dynamic latent and appearance slots at the occupied cells of t.
2. Decode static, far-field and dynamic features with their occupancy hierarchies; inject the appearance codes at 128^3.
3. Predict surfels using the source camera at t for the base configuration. Jointly rasterize the three layers in the target view to obtain colour, depth and opacity.
4. Decode the texture codes into texture frames $\hat{I}_\tau$ and render source-view depths from the same state for the stored frames. Apply the projection and depth-consistency test in Eq. 8, with temporal blending for dynamic texture.
5. Composite the textured rendering over the mean-colour background.

A.2. Observation preprocessing

Depth and points. We run pose-conditioned Depth Anything 3 on overlapping 32-frame chunks of the full video. Consecutive chunks are aligned in scale using the median depth ratio on their overlap. Normals are computed from depth gradients, and sky pixels flagged by the depth model are excluded from the point cloud.

See-through test. Let $(\mathbf{x}', \zeta')$ be the projection of $\mathbf{p}_t(\mathbf{x})$ into another frame t'. The point is marked dynamic if, for some $|t - t'| \in \{1, 2, 4, 8, 16\}$,

$$\frac{D_{t'}(\mathbf{x}') - \zeta'}{\max(\zeta', 0.05\,\mathrm{m})} > 0.08. \tag{7}$$

We set $D_{t'} = \infty$ on sky. Connected components smaller than 48 pixels are removed, and the mask is dilated by four pixels.

Object masks. RMBG-2.0 (BiRefNet [63]) provides foreground components. An 8-connected component c is selected if it contains at least $\max(16, 0.02|c|)$ dynamic pixels from the see-through test and covers at most 35% of the image. Selections propagate forward and backward to components overlapping a selected region by at least 30% of their area. Selected regions are dilated by two pixels.

Spatial layout. The scene cube has side length $s = \mathrm{clip}(3\tilde{d}, 4\,\mathrm{m}, 32\,\mathrm{m})$, where $\tilde{d}$ is median camera-to-point distance. Its centre is the median of the nearest 30% of points. A 512^3 grid gives voxel widths of approximately 0.8–6.3 cm. Static points outside the cube are contracted into the far-field shell (Eq. 2) and voxelized on a second 512^3 grid over $[-2, 2]^3$.

Voxel features. Each voxel has nine base features: mean RGB, mean normal and mean within-voxel point offset. For each of eight octants we append the difference between its mean RGB and the voxel mean, plus an occupancy flag.

A.3. Appearance and slot modules

$\mathbf{W}_d$ and the octant-feature input projection are zero-initialized and scaled by a fixed gain $\gamma = 10$, so both residual pathways start at zero. $\mathbf{W}_e$ uses the default initialization. Gradients in the gained layers are scaled by $1/\gamma^2$ before clipping.

The temporal modules use width 256 and eight attention heads. Two learned queries pool the observed tokens of each cell. Three self-attention blocks mix all slots, followed by projection to the stored channel dimension. Three cross-attention blocks decode each (u, t) query. Latent slots have 32 channels; appearance slots have 4. Both receive the normalized feature-matching losses in Appendix B.

A.4. Stored-texture reprojection

Given target depth $\hat{d}_c$, a pixel is back-projected to $\mathbf{X} = \pi_c^{-1}(\mathbf{x}, \hat{d}_c(\mathbf{x}))$ and projected into stored frame τ as $(\mathbf{y}, \zeta) = \pi_\tau(\mathbf{X})$, where π denotes calibrated camera projection and ζ

Table 9. **Stored-state schema.** Features occupy sparse cells on the indicated grids. Static and far-field features are shared across the clip, while dynamic slots are stored on the temporal union of occupied cells. Dynamic occupancy retains the hierarchy at each observed timestamp. The complete state includes stored texture and background information; cameras and shared network weights are external to the state-size accounting.

Component	Grid	Stored form	Decoding role
Static and far-field latent $\mathbf{z}$	32^3	32 channels per occupied cell, once per clip	Shared spatial features
Dynamic latent slots	32^3	Two 32-channel vectors per union cell	Time-varying features read at (u, t)
Static and far-field appearance $\mathbf{a}$	128^3	Four channels per occupied cell, once per clip	Fine-grid appearance injection
Dynamic appearance slots	128^3	Two 4-channel vectors per union cell	Time-varying appearance read at (u, t)
Sparse occupancy	32^3–512^3	Static and far-field hierarchies and per-frame dynamic hierarchies	Sparse upsampling at an observed time
Stored texture	256^2 images	Full frame 0 and 16-channel codes on dynamic 8×8 tiles at $4, 8, \ldots, 28$	Colour sampling through decoded geometry
Background	–	Source-mean RGB	Pixels without surfels

is the depth of $\mathbf{X}$ in frame τ. We accept the sampled colour $I_\tau(\mathbf{y})$ when

$$|\zeta - \hat{d}_\tau(\mathbf{y})| < 0.03\zeta + 0.02\,\mathrm{m}, \tag{8}$$

where $\hat{d}_\tau$ is rendered from the same decoded state in the source view of frame τ. Otherwise the surfel colour is retained. Static pixels take colour from the first frame. Pixels where the dynamic layer alone has opacity above 0.5 take colour from the first frame only at $t = 0$. At other times they use the dynamic texture below.

Dynamic stored texture. Texture codes are stored at $t = 4, 8, \ldots, 28$. A dynamic pixel at time t blends the decoded texture frames $\hat{I}_{\tau^-}$ and $\hat{I}_{\tau^+}$, with $\tau^- \le t < \tau^+$, by temporal proximity. Only samples that satisfy Eq. 8 and have target opacity above 0.5 are used; source depth is rendered at each stored timestamp. Frame 0 serves as τ^- for $t < 4$, and only τ^- is available for $t > 28$. Texture is applied before background compositing.

B. Training details

The supervised timestamps are $\{0, t, \tau^-, \tau^+\}$. If two coincide or τ^+ falls outside the window, random timestamps fill the gap. Before stored texture is introduced, all four timestamps are random. The four target cameras are drawn at random from the nine non-source cameras. The loss is

$$\begin{aligned}\mathcal{L} &= \tfrac{1}{4}\textstyle\sum_q \mathcal{L}_q + 0.5\,\mathcal{L}_{\Delta t} + \mathcal{L}_{\text{lat}} + \mathcal{L}_{\text{lat}}^{\text{app}} \\ &\quad + 0.1\,\mathcal{L}_{\text{occ}} + 10^{-6}\,\mathcal{L}_{\text{KL}}, \end{aligned} \tag{9}$$

$$\begin{aligned}\mathcal{L}_q &= \mathcal{L}_{\text{rgb}}^{\text{src}} + \tfrac{1}{4}\textstyle\sum_v \mathcal{L}_{\text{rgb}}^{v} \\ &\quad + 0.3\left(\mathcal{L}_d^{\text{src}} + \tfrac{1}{4}\textstyle\sum_v \mathcal{L}_d^{v}\right) + 0.05\,\mathcal{L}_n, \end{aligned} \tag{10}$$

where v runs over the four supervising target cameras. We use

$$\mathcal{L}_{\text{rgb}} = \|\hat{I} - I\|_1 + 0.2(1 - \text{SSIM}) + \lambda_{\text{LPIPS}}\,\text{LPIPS}_{\text{VGG}}$$

on images composited over the mean-colour background after stored-texture replacement.

$\mathcal{L}_d$ is an L_1 loss on log depth against the per-camera depth estimate, restricted to pixels inside the scene cube and covered by surfels. $\mathcal{L}_n$ is the 2DGS normal-consistency term [12]. $\mathcal{L}_{\Delta t} = \text{mean}_{a<b}\|(\hat{I}_a - \hat{I}_b) - (I_a - I_b)\|_1$ penalizes temporal differences that do not match the ground truth. $\mathcal{L}_{\text{lat}}$ and $\mathcal{L}_{\text{lat}}^{\text{app}}$ are the relative squared errors between slot-decoded and per-frame latents and appearance codes. For the appearance code, the loss uses a random subset of 65,536 rows. $\mathcal{L}_{\text{occ}}$ and $\mathcal{L}_{\text{KL}}$ are the occupancy and KL terms of TRELLIS.2 [16, 53].

Optimization. We use AdamW [31] ($\beta = (0.9, 0.95)$, no weight decay) and global gradient-norm clipping at 1.0. Each stage warms up linearly to a peak learning rate of 3×10^{-5} (2×10^{-5} for the far-field shell) and decays it to 10^{-5} with a cosine schedule. The slot modules use a $10\times$ learning rate. λ_{LPIPS} is 0.2 before stored texture is introduced and 0.6 afterwards. Training uses one clip per GPU on eight NVIDIA A800 GPUs. The autoencoder has 836M parameters, almost all in the TRELLIS.2 backbone. The texture codec has 7.3M parameters and is trained with batches of 16 frames, a peak learning rate of 2×10^{-4} and cosine decay to 2×10^{-5}.

B.1. Design parameters

C. Representation-size accounting

Occupancy is represented as a 32^3 bitmap followed by 8-bit child masks at four finer octree levels. Its cost is the

Table 10. **Refresh interval of the texture codes** (404 random-window clips). All rows use the same trained weights and texture codec; the first source frame is always stored. #Floats is the mean state size per 32-frame clip.

Interval	#Floats↓	PSNR↑	LPIPS↓	Dyn. LPIPS↓
1	0.54M	17.54	0.364	0.260
2	0.45M	17.53	0.366	0.262
4 (Sparc4D)	0.40M	17.51	0.367	0.264
8	0.37M	17.47	0.368	0.268

Table 11. **State size of Sparc4D** in thousands of 32-bit-equivalent values per 32-frame clip (random-window protocol; first-32 values in parentheses). Occupancy uses empirical entropy divided by 32. The first source frame adds $256^2 \times 3$ values; the texture-code cost depends on the dynamic area.

Component	Mean	Median
Static + far-field latent	15.1	12.6
Dynamic temporal slots	22.7	16.3
Static + far-field appearance code	28.7	24.1
Dynamic appearance slots	56.7	38.5
Occupancy, static + far-field	6.7	5.5
Dynamic occupancy (32f)	27.8	19.9
State without stored texture	157.8 (149.7)	124.4 (124.7)
Subtotal with first source frame	354.4 (346.3)	321.0 (321.4)
Texture codes, every 4th frame	42.9 (42.0)	39.0 (37.3)
Sparc4D (complete state)	397.3 (388.3)	363.2 (356.6)

zeroth-order empirical entropy of these symbols. The code-tile masks are not stored, because the decoder recomputes them from the dynamic geometry. Scene-layout metadata such as cube origin and scale are omitted.

Per-frame features. The per-frame row of Table 2 bypasses the slots at inference and stores dynamic latent and appearance features at every timestamp, at a cost of

$$N_{\text{pf}} = 32 \sum_{t=1}^{T} |\mathcal{U}_t| + 4 \sum_{t=1}^{T} |\mathcal{U}_t^{128}|, \tag{11}$$

where $\mathcal{U}_t$ and $\mathcal{U}_t^{128}$ are the occupied dynamic cells at time t on the 32^3 latent and 128^3 appearance grids, respectively. With temporal slots, the corresponding cost is

$$N_{\text{slot}} = 64|\mathcal{U}| + 8|\mathcal{U}^{128}|, \tag{12}$$

where $\mathcal{U}$ and $\mathcal{U}^{128}$ are the temporal unions of occupied dynamic cells.

For the combined time-varying features in Table 3, the 10th–90th percentile range of the per-clip compression ratio is 3.3×–7.4×.

For each baseline, we report the fewest 32-bit values from which any of the 32 input timestamps can be rendered. MoVieS predicts 1,605,632 Gaussians for 32×224^2 input pixels, each with 14 static parameters and 4×32 query-time motion parameters, which gives 228M values. For 4DGT we count the predicted Gaussian parameters. For C4G, the 2,048 Gaussians decoded at each of the 32 timestamps (0.92M values) are fewer than its latent tokens, so we count them.

D. Simple alternatives to the 4D state

D.1. Temporal compressors

We replace the temporal slots with fixed temporal models, fitted by least squares to the per-frame features of each occupied cell, for both the dynamic latents and the dynamic appearance codes. Per-frame occupancy, the decoder and stored texture are unchanged, and no network is retrained. A two-term DCT [29, 46], a linear function of time [24, 29] and two keyframes with linear interpolation [18] store two vectors per cell, as two slots do. The per-cell mean stores one vector, and a four-term DCT stores four. The low-rank model stores a per-cell mean and per-frame coefficients on a basis shared within the clip, in the spirit of shared motion bases [49]. Its rank is chosen to approximate the slot storage. Table 12 reports paired differences to the slots. On the first-32 protocol, the per-cell mean, DCT and linear models are also within 0.02 dB of the slots in full-image and dynamic-region PSNR.

Table 12. **Temporal compressors** (404 random-window clips). Dyn. #Floats is the mean number of stored dynamic latent and appearance values per clip. Latent error is the relative squared error of the decoded dynamic latents. Δ is the paired per-clip difference in dynamic-region PSNR to the temporal slots (mean ± 95% CI), with stored texture and for the state alone.

Compressor	Dyn. #Floats	Latent error	Δ, with texture (dB)	Δ, state only (dB)
Temporal slots (Sparc4D)	79.4K	0.058	–	–
Per-cell mean	39.7K	0.184	-0.00 ± 0.01	-0.06 ± 0.03
DCT, 2 terms [29, 46]	79.4K	0.139	-0.01 ± 0.01	-0.04 ± 0.02
DCT, 4 terms [29, 46]	158.8K	0.100	-0.01 ± 0.02	-0.02 ± 0.02
Linear in time [24, 29]	79.4K	0.140	-0.01 ± 0.01	-0.04 ± 0.02
Low-rank, shared basis [49]	94.9K	0.107	-0.02 ± 0.01	-0.04 ± 0.02
Two keyframes [18]	79.4K	0.265	-0.02 ± 0.04	-0.11 ± 0.04
Per-frame features	354.5K	0	-0.02 ± 0.01	-0.00 ± 0.04

D.2. Geometry for the same stored texture

Rows with raw texture in Table 13 use the same stored pixels: the first source frame and the raw dynamic-region pixels of every fourth frame, reprojected and blended as in Sparc4D. Only the geometry differs. *Input voxels* renders the static, far-field and per-frame dynamic input voxels as

camera-facing disks with their mean colour, at 512^3 or after re-voxelization at 256^3 and 128^3. *Stored RGB-D frames* back-projects the stored pixels with their DA3 depth: all pixels of the first frame and the dynamic-region pixels of every fourth frame. Disk sizes are selected on six training clips. Voxel rows count six values per voxel and the occupancy entropy, and RGB-D pixels count four values.

Table 13. **Geometry for the same stored texture** (404 random-window clips). Texture is the learned codes of Sparc4D, raw stored pixels, or none. #Floats is the mean total per clip, including stored texture.

Geometry	Texture	#Floats	PSNR↑	SSIM↑	LPIPS↓	Dyn. LPIPS↓
Learned 4D state (Sparc4D)	Codes	0.40M	17.51	0.524	0.367	0.264
Learned 4D state	Raw	0.67M	17.53	0.524	0.366	0.263
Input voxels, 512^3	Raw	2.42M	17.15	0.505	0.388	0.296
Input voxels, 256^3	Raw	1.34M	16.96	0.497	0.405	0.317
Input voxels, 128^3	Raw	1.00M	16.69	0.484	0.438	0.357
Stored RGB-D frames	Raw	0.74M	17.02	0.492	0.397	0.308
Learned 4D state	None	0.16M	16.99	0.496	0.466	0.346
Input voxels, 512^3	None	1.85M	16.61	0.481	0.479	0.377

E. Runtime and memory

Table 14 reports runtime on one A800 GPU with batch size 1, as the median over 24 test clips. Each clip has 32 input frames and is rendered at 32 timestamps from 10 cameras, giving 320 images at 256^2. MoVieS and 4DGT decode inside their renderers, and C4G decodes all timestamps during encoding. Timings for Sparc4D start from the DA3 depth maps. Depth estimation is not included and adds 3.42 s per clip.

Table 14. **Runtime and memory per clip.** For Sparc4D, encoding includes RMBG-2.0 foreground masks and voxelization, and rendering includes texture-code encoding and decoding and stored-texture reprojection.

Method	Encode	Decode	Render	Per image	Memory	Params
MoVieS (bf16)	2.13 s	–	2.42 s	7.6 ms	15.7 GB	1041M
4DGT	1.10 s	–	7.55 s	23.6 ms	9.8 GB	1223M
C4G	2.29 s	–	0.09 s	0.3 ms	5.3 GB	1014M
Sparc4D (fp32)	1.21 s	0.24 s	0.99 s	3.1 ms	5.3 GB	1064M

F. Evaluation details

F.1. Split and camera separation

We compute a DINOv2 environment feature for each scene and cluster the 3,400 scenes with k-means ($k = 120$). Test scenes are drawn from clusters assigned to the test side, and each has cosine similarity below 0.62 to every scene in the remaining clusters. This gives 101 test scenes, each rendered at four focal lengths (404 clips).

The remaining 3,299 scenes (13,196 clips) form the training set.

The four focal lengths are 18, 24, 35 and 50 mm. The ten cameras coincide at frame 0 and separate over time. Measured over 404 test clips, mean pairwise separation is 0.93 m at frame 31 and 2.10 m at frame 80. In first-32, 70% of target images are within 0.5 m of the source camera; in random windows, 35% are. The two protocols therefore cover different view-separation regimes.

F.2. Visibility masks and image metrics

MultiCamVideo provides no co-visibility masks. We construct a shared mask from estimated depth and foreground segmentation, independently of each reconstruction method. A target pixel is back-projected using its estimated depth. Depth scale is aligned to the source using a median depth ratio before projecting the point into source frames.

A background pixel is accepted if at least one source frame sees a background surface at the projected position with depth agreement within 10%. An RMBG foreground pixel is accepted only through the source frame at the same time. This restriction avoids treating a moving point as static across frames. Sky and points beyond 40 m are checked by direction. The resulting masks cover on average 83% of target pixels for first-32 and 64% for random windows.

For mask M and RGB intensities in $[0, 1]$, we use

$$\mathrm{mPSNR} = -10 \log_{10} \frac{\sum_{\mathbf{x}} M(\mathbf{x}) \|\hat{I}(\mathbf{x}) - I(\mathbf{x})\|^2}{3 \sum_{\mathbf{x}} M(\mathbf{x})}. \quad (13)$$

mSSIM and mLPIPS average their spatial metric maps inside the mask. Images with less than 1% observed pixels are omitted from masked averages, about 2% of random-window images. Masked and uniform-fill scores use stored 8-bit renderings.

For region analysis, current-frame dynamic input voxels are rendered as camera-facing disks into each target camera. We threshold opacity at 0.5 and dilate by two pixels. The static region lies inside the scene cube. Dynamic-region LPIPS uses the region's bounding box, padded by eight pixels and resized to 128^2.

F.3. Baseline configurations

MoVieS receives all 32 frames at 224^2, together with ground-truth cameras and timestamps. 4DGT receives all 32 frames at 560^2 and ground-truth cameras. Its timestamps use frame index divided by five rather than the dataset's physical rate of 15 fps. We select this setting by source-view reconstruction quality.

C4G is pose-free and receives 12 frames spread uniformly over the window at 224^2, matching its training context count. We align its arbitrary reconstruction scale to the estimated depth of its first context frame using a median

ratio. Target cameras are expressed relative to that frame. The fine-tuned model also keeps C4G's architecture and 12-frame input.

F.4. DyCheck preprocessing

We use the *apple*, *block*, *paper-windmill*, *space-out*, *spin*, *teddy* and *wheel* sequences. The handheld video supplies 32 input frames from every second timestamp of a non-overlapping 64-timestamp window. At 30 fps this gives the 15 fps input sampling used in training. Targets are fixed-camera images present in the official validation split at those input times. Windows with no target image are dropped.

We centre-crop the half-resolution 360×480 images to 360^2 and resize to 256^2. Images are resampled to the intrinsics of the first handheld frame in each window. Official co-visibility masks use nearest-neighbour resampling. Source depth and foreground masks are prepared as for MultiCamVideo. The baselines use Appendix F.3' settings; all receive known cameras except pose-free C4G.

Table 15. **mPSNR per DyCheck sequence.** The texture-free variant is an ablation of Sparc4D.

Sequence	MoVieS	w/o stored texture	Sparc4D
apple	*15.54*	**15.62**	15.38
block	14.84	**15.37**	*15.06*
paper-windmill	**15.79**	*14.73*	14.44
space-out	*14.16*	**14.36**	14.07
spin	**13.49**	*13.48*	13.11
teddy	11.38	**12.76**	*12.23*
wheel	**12.94**	*11.72*	11.46

F.5. Neu3D preprocessing

We evaluate six Neu3D [21] scenes using 24 non-overlapping 32-frame windows. For each window, a fixed central camera supplies the monocular input. At the same timestamps, the targets are cam00 and eight cameras sampled at approximately uniform index intervals, for nine target cameras in total. We report full-image PSNR, SSIM and LPIPS over all nine targets, and PSNR separately for cam00. Cam00 is the conventional Neu3D test camera and is approximately 10 cm from the input camera. All methods use the settings in Appendix F.3.

G. Additional qualitative comparisons

Figures 5–8 show more examples with the layouts of Figs. 2 and 3.

H. Depth consistency

MultiCamVideo does not provide ground-truth depth. We compare rendered target-view depth with per-camera DA3 estimates using AbsRel, RMSE and δ_1, the fraction of pixels whose depth ratio is within 1.25 [8]. We also report median-scaled AbsRel and δ_1. Each method is evaluated where it renders a surface and the reference is valid; coverage is reported alongside the errors.

Table 16. **Rendered depth vs. the per-camera depth estimate** at target views (test set, per-image average). "aligned": per-image median scaling. C4G-FT: C4G fine-tuned on MultiCamVideo.

					aligned		
	Method	AbsRel↓	δ_1 ↑	RMSE↓	AbsRel↓	δ_1 ↑	cov.
First-32	4DGT	0.624	0.141	8.56	0.344	0.621	0.92
	C4G	0.505	0.538	6.34	0.476	0.586	0.91
	C4G-FT	*0.267*	*0.592*	*5.16*	0.291	0.598	**0.98**
	MoVieS	0.561	0.513	6.21	*0.250*	*0.717*	0.92
	Sparc4D	**0.174**	**0.857**	**2.45**	**0.127**	**0.885**	*0.93*
Random	4DGT	0.640	0.155	8.40	0.370	0.598	0.81
	C4G	0.543	0.522	6.12	0.450	0.596	*0.84*
	C4G-FT	*0.356*	*0.524*	*5.43*	0.346	0.545	**0.97**
	MoVieS	0.615	0.505	5.84	*0.272*	*0.704*	0.75
	Sparc4D	**0.270**	**0.790**	**2.81**	**0.179**	**0.830**	0.77

Rendered depth from Sparc4D is more consistent with the DA3 reference than the evaluated baselines. Because DA3 also supplies our input depth and depth supervision, this comparison measures agreement with DA3.

I. Downstream manipulation protocol

Setup. The autoencoder receives 4,000 adaptation steps on ManiSkill scenes before policy training. Four 256^2 RGB-D views, sampled on a ring around the workspace, are fused into a fixed 2.4 m cube. Static features pool the first three frames of each observation window. Simulator depth and robot-link silhouettes are used; SAM 2 tracks objects causally from boxes obtained by clustering non-robot points above the table, without object-pose or object-segmentation labels. The policy also observes the end-effector pose and gripper state. It is trained for 60,000 steps with batch size 64. It predicts eight-step action chunks with ten Euler steps and replans after each chunk. The additional 2,000 demonstrations randomize motion-planning waypoints and initial joint-motion prefixes.

Evaluation. Success is measured within 200 control steps over 100 initial states and three sampling seeds per trained policy. Li et al. [26] report results with fixed 512^2 cameras and joint-position control; ours use randomized cameras and end-effector increments.

Scene size. Our counts average 306 held-out windows: 425.9 static and 235.5 dynamic cells per timestamp, or

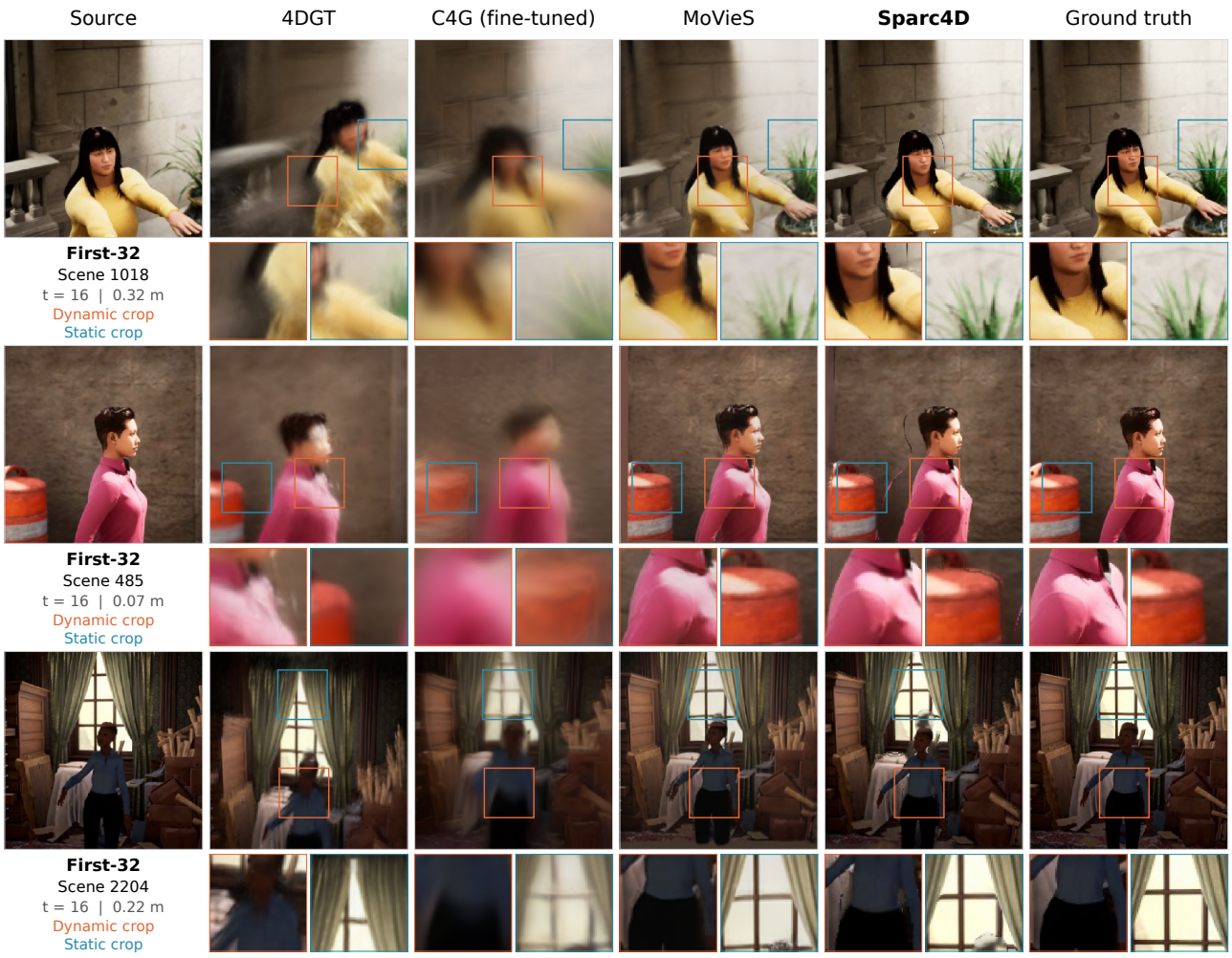


Figure 5. **Additional MultiCamVideo examples, first-32 protocol.** Layout and methods as in Fig. 2.

661.4 in total. The three-frame policy context contains 1,132.5 cells. The same encoder and reference windows are used for both data budgets. The approximately 5,700 cells for Li et al. come from three initial states with their fixed cameras. These counts include scene-latent cells only.

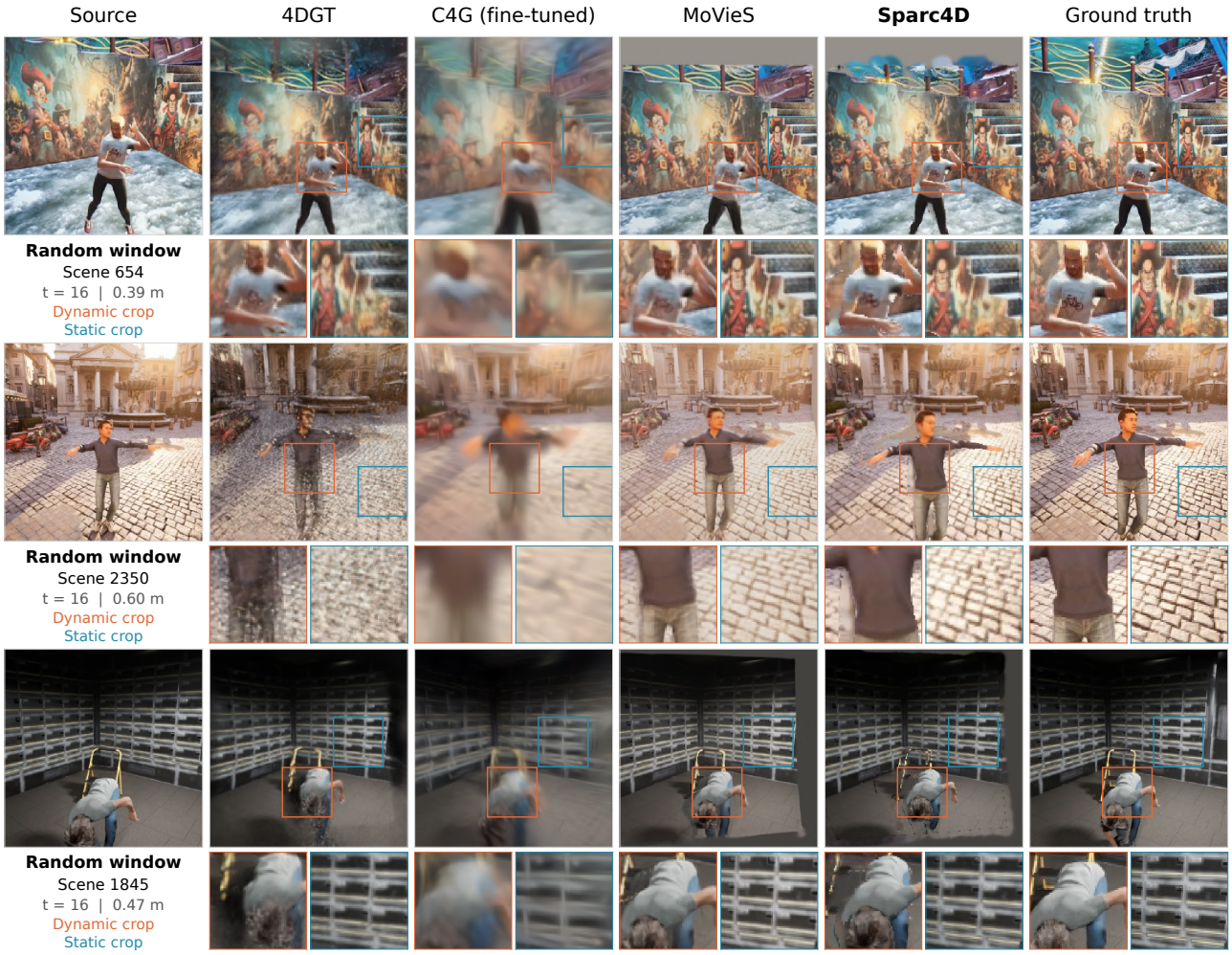


Figure 6. **Additional MultiCamVideo examples, random-window protocol.** Layout and methods as in Fig. 2.

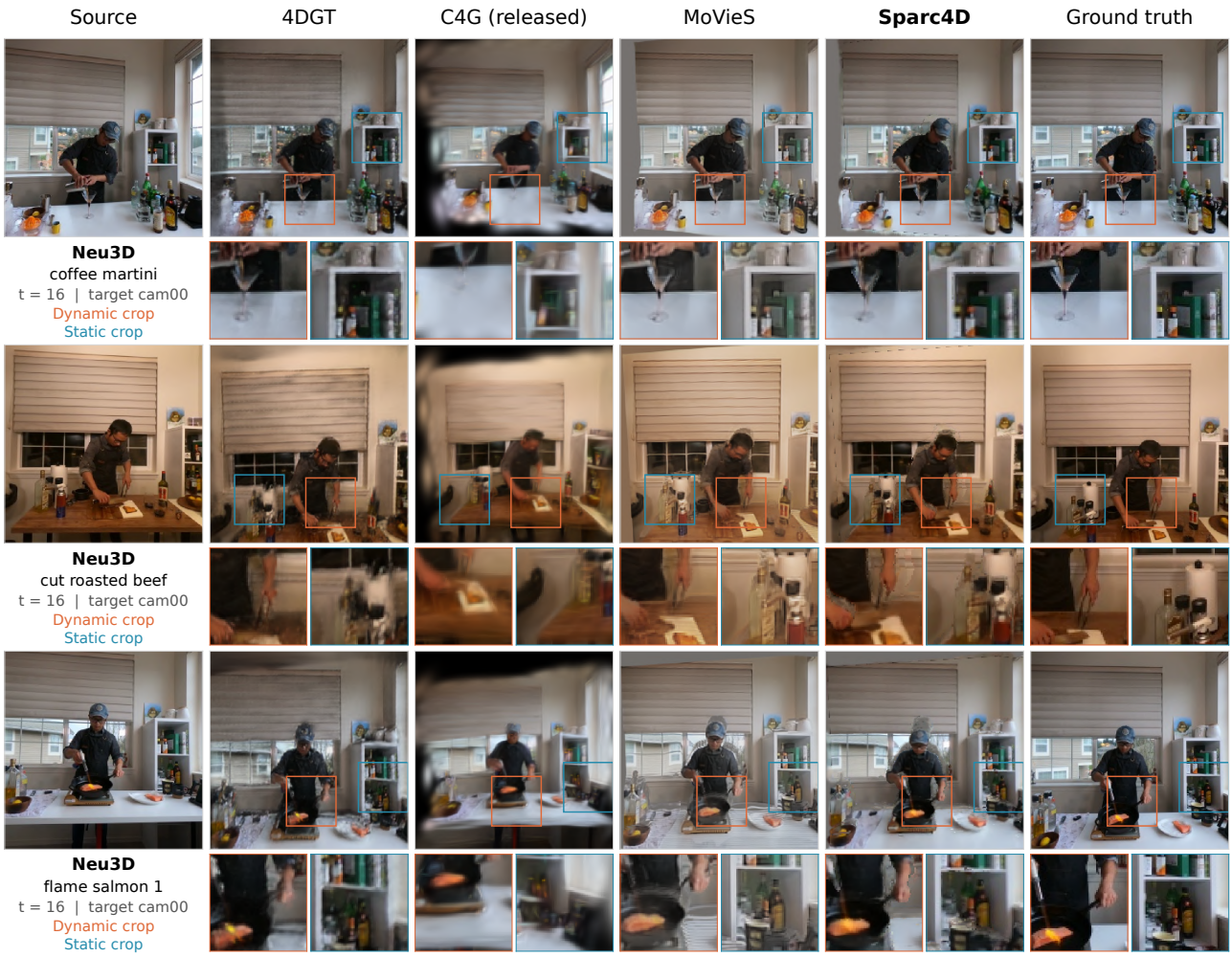


Figure 7. **Additional zero-shot examples on Neu3D.** Layout and methods as in Fig. 3.



Figure 8. **Additional zero-shot examples on Neu3D (continued).** Layout and methods as in Fig. 3.